

Graph-Guided and Adaptive KV Cache Prefetching for Retrieval-Augmented Generation: A Hit-Aware, Cross-Architecture Evaluation Across Model Scales, Model Families, and Datasets

[Author details withheld for proofreading]

Abstract

Retrieval-Augmented Generation (RAG) systems incur substantial latency when a query requires a document chunk that is not already resident in the Key-Value (KV) cache, forcing an expensive re-encode. Existing prefetching strategies rely primarily on cosine similarity between chunk embeddings, an assumption that systematically fails for structural and multi-hop reasoning queries whose retrieval relationships are determined by document topology rather than embedding proximity. We propose a hybrid chunk graph that augments semantic similarity edges, constructed via top- M nearest-neighbor ranking rather than a fixed similarity threshold, with structural positional edges, and retrieve prefetch candidates via two-hop traversal. We further introduce an adaptive variant that learns a query-type-specific balance between semantic and structural edge weight from prior measurements, rather than using a single fixed weighting for every query. Central to this work is a correction to how prefetching latency must be measured: a cache hit has to be charged the cost of a cache lookup rather than a fresh re-encode, or no prefetching policy can ever be measured as outperforming an uncached baseline regardless of its actual hit rate. Under this hit-aware measurement, our primary evaluation now sweeps the full Qwen2.5 family (1.5B, 3B, and 7B-Instruct) across all four LongBench datasets we study (HotpotQA, 2WikiMultiHopQA, MuSiQue, and MultiFieldQA-en) on a single RTX 5060 Ti, superseding our earlier single-scale and single-dataset sweeps. We additionally test cross-architecture generalization on Llama-3.2 (1B, 3B) and Gemma-3-1B, with partial policy and dataset coverage, and we report a preliminary closed-source answer-quality check (Gemini 2.5 Flash-Lite and Llama-3.3-70B via Groq) asking whether a policy’s latency advantage carries over to downstream QA accuracy. We find that the adaptive policy’s advantage over cosine similarity is now the single most consistent result in this study: it reaches significance from $K=6$ onward and holds at every larger K , at *every* tested scale (1.5B/3B/7B), pooled across all four datasets. The structural-query crossover point is exactly $K=3$ in every one of the six models we evaluate, across three different model families, the most reproducible effect in the paper. Multi-hop queries again show a real but comparatively noisier effect. Cross-architecture testing on Llama and Gemma reproduces the qualitative pattern (structural favors the graph almost immediately, semantic always favors cosine) even where coverage is partial. Finally, our closed-source QA-quality check finds the opposite ranking from the latency results in most configurations: cosine similarity matches or leads graph and adaptive on F1/EM, and the adaptive policy trails both alternatives in every configuration tested. We report this candidly as a caveat rather than smoothing it over: a policy’s cache-latency advantage and its downstream generation-quality behavior are not the same objective, and our evidence here (single dataset, $n=53\text{--}60$ per configuration) is preliminary.

1 Introduction

Retrieval-Augmented Generation (Lewis et al., 2020) grounds LLM outputs by appending retrieved document chunks to the model context at inference time. Modern inference frameworks such as vLLM (Kwon et al., 2023) maintain a KV cache of pre-computed attention tensors to enable rapid reuse when the same prefix appears in multiple requests. A single user query, however, typically requires several related chunks in sequence: accessing a primary chunk often triggers a cascade of secondary retrievals, most of which miss the cache and force a full re-encode.

The dominant approach to reducing these misses is cosine-similarity prefetching: when a chunk is accessed, the top- K most semantically similar chunks are pre-loaded. This assumes semantic similarity is the primary predictor of co-access probability, an assumption that fails for two query categories common in real RAG workloads. Structural queries require sequentially adjacent document context – a methodology section and its immediately following results section may be semantically dissimilar yet always co-accessed. Multi-hop queries require evidence from passages that are related both semantically and structurally across document regions, the defining characteristic of datasets such as HotpotQA and 2WikiMultiHopQA.

We address this with a hybrid chunk graph combining semantic and structural edges, and an adaptive variant that learns how much to weight each signal per query type rather than fixing that balance in advance. A substantial part of this work concerns how such a system must be measured in the first place, and, in this revision, how far its conclusions generalize once the evaluation is widened along three further axes: model scale within a single family, model family itself, and the downstream answer quality actually produced by the retrieved context rather than only the latency of retrieving it.

Contributions. This paper makes six contributions. First, we identify and correct a latency measurement methodology that, left uncorrected, makes prefetching quality appear unrelated or even inversely related to cache hit rate (§3.6). Second, we propose a hybrid chunk graph using top- M semantic neighbor construction, structural adjacency, and two-hop candidate retrieval, which resolves a coverage failure of fixed-threshold semantic graphs (§3). Third, we introduce an adaptive, query-type-specific edge weighting learned from prior hit-aware measurements, which now significantly outperforms cosine similarity from $K \approx 6$ onward at every one of three tested Qwen2.5 scales, pooled across all four datasets we study (§5.1). Fourth, we extend the evaluation to three additional open-weight model families – Llama-3.2-1B, Llama-3.2-3B, and Gemma-3-1B – and show the structural-query effect reproduces at exactly $K=3$ in every model tested, the most consistent finding in this paper (§5.2). Fifth, we report a closed-source answer-quality evaluation using Gemini 2.5 Flash-Lite and Llama-3.3-70B (via Groq), which is, to our knowledge, the first check in this line of work of whether a prefetching policy’s latency advantage corresponds to any change – for better or worse – in the quality of the answer eventually produced (§5.6). Sixth, having pooled the K-sweep across all four datasets rather than evaluating HotpotQA alone, we revisit and revise an earlier hit-rate-plateau claim that does not hold once the full dataset pool is considered (§5.4).

2 Related Work

KV cache management. PagedAttention (Kwon et al., 2023) introduced virtual-memory paging for KV cache blocks, enabling efficient memory management in vLLM. LMCache (Liu et al., 2024b) extends prefix caching across requests by detecting shared prompt prefixes and, through its CacheBlend-style blending mode, supports reusing independently computed chunk KV states even when they do not appear as a shared prefix, at the cost of a partial recomputation to correct for

missing cross-chunk attention. CacheGen (Liu et al., 2024a) explores KV compression and streaming. None of these systems address predicting which chunks will be needed before they are explicitly requested, the problem we address here; our findings on when a hit is actually cheaper than a miss are directly relevant to evaluating any of them.

Predictive prefetching. Hardware prefetching exploits access pattern regularity to hide memory latency (Chen and Baer, 1995). In the LLM context, Jin et al. (2024) propose caching retrieved documents across requests. Our work differs by building a graph over intra-document chunk relationships for proactive prefetching within a single inference session, without requiring cross-request history.

Graph-based RAG. GraphRAG (Edge et al., 2024) and related methods (Hu et al., 2024) construct knowledge graphs over document entities to improve retrieval quality. Our graph is a lightweight chunk-level structure for prefetch prediction rather than entity reasoning, and prioritizes inference latency rather than retrieval recall.

Multi-hop reasoning benchmarks. HotpotQA (Yang et al., 2018), 2WikiMultiHopQA (Ho et al., 2020), and MuSiQue (Trivedi et al., 2022) demonstrate that real questions frequently require multi-document evidence chains, motivating our multi-hop query distribution and dataset selection. LongBench (Bai et al., 2024) provides the unified benchmark framework used in our evaluation.

Model families evaluated. Our cross-architecture check draws on three open-weight families – Qwen2.5 (Qwen Team, 2024), Llama 3 (Llama Team, 2024), and Gemma 3 (Gemma Team, 2025) – and two closed-source models accessed via API for the answer-quality check – Gemini (Gemini Team, 2024) and Llama-3.3-70B served via Groq.

3 Method

3.1 Problem Formulation

Let $C = \{c_1, \dots, c_N\}$ be document chunks with embeddings $e_i \in \mathbb{R}^d$ and KV states κ_i computed by the LLM over each chunk’s tokens. At inference time, query q triggers access to primary chunk c_p . Our goal is a prefetch function

$$P(c_p, K) \rightarrow \{c_{j_1}, \dots, c_{j_K}\} \quad (1)$$

selecting the K chunks most likely to be accessed next, so their KV states can be pre-loaded before they are formally requested.

3.2 Document Processing

Documents are chunked with LangChain’s `RecursiveCharacterTextSplitter` using a chunk size of 500 characters and an overlap of 50 characters. Each chunk is encoded with `all-MiniLM-L6-v2` (Reimers and Gurevych, 2019) into 384-dimensional embeddings. This is unchanged across every model and dataset in this revision – the graph is built once per corpus from these embeddings, independent of which downstream LLM ultimately consumes the prefetched chunks.

3.3 Hybrid Chunk Graph Construction

An earlier version of this graph connected two chunks whenever their cosine similarity exceeded a fixed threshold $\tau = 0.55$, or whenever they were immediately sequentially adjacent. This construction starves a chunk of semantic edges entirely whenever the corpus simply does not contain a sufficiently similar neighbor above τ , regardless of how useful a weaker connection to that chunk

might have been – a coverage failure that we found was substantially responsible for the fixed-weight graph policy’s inconsistent performance in preliminary experiments.

We instead construct the semantic component of the graph via top- M nearest-neighbor ranking ($M=20$): every chunk is connected to its twenty most similar neighbors by rank rather than by an absolute similarity cutoff, and the resulting directed relation is symmetrized in both directions. This guarantees baseline semantic coverage for every node independent of the corpus’s overall similarity distribution. We retain a semantic threshold, now lowered to $\tau = 0.50$, and immediate sequential adjacency ($|i - j| = 1$) as a secondary structural gate, so that a chunk pair is connected if

$$j \in \text{topM}(i) \vee i \in \text{topM}(j) \vee \cos(e_i, e_j) \geq \tau \vee |i - j| = 1. \quad (2)$$

Edge weights combine both signals,

$$w_{ij} = \alpha \cdot s_{\text{sem}}(i, j) + \beta \cdot s_{\text{struct}}(i, j), \quad (3)$$

with $s_{\text{sem}}(i, j) = \cos(e_i, e_j)$ if the pair qualifies as a semantic edge and 0 otherwise, and $s_{\text{struct}}(i, j) = 1/(1 + |i - j|)$. The fixed-weight graph policy uses $\alpha=0.70$, $\beta=0.30$ throughout. In-degree is capped at 100 for our main results; we separately verified, by re-running the adaptive policy with the cap lowered to 10, that this parameter does not materially affect any result reported here (§5.5). Across every corpus in this revision – including the newly added Llama and Gemma runs – the realized average node degree before and after pruning is essentially identical (typically 27–29, see log excerpts in the supplementary evaluation logs), confirming the cap does not bind at $M=20$ regardless of which downstream model consumes the graph.

Two-hop retrieval. Prefetch candidates for a query chunk c_p are drawn not only from c_p ’s direct neighbors but also from neighbors of those neighbors, since a chunk that is two structural or semantic hops away is frequently still a relevant prefetch target and restricting retrieval to one hop under-utilizes the graph. A naive product of hop weights, $w_1 \cdot w_2$, disproportionately penalizes a genuine but modest two-hop structural chain relative to a two-hop path through two moderately strong semantic edges. We instead score a two-hop candidate as

$$\text{score} = 0.9 \cdot \min(w_1, w_2) + 0.1 \cdot \max(w_1, w_2), \quad (4)$$

which remains bottleneck-scored by the weaker of the two hops while still distinguishing a strong→strong path from a strong→weak path. Candidates are ranked by this score and the top- K retained.

3.4 Adaptive Query-Type-Specific Weighting

The fixed weighting $(\alpha, \beta) = (0.70, 0.30)$ is a single compromise applied to every query regardless of type. We instead learn a separate (α, β) per query type ($\alpha + \beta = 1$), re-ranking the same two-hop candidate set described above, chosen by a grid search over $\alpha \in \{0.35, 0.40, \dots, 0.80\}$ minimizing expected latency under the hit-aware cost model of §3.6:

$$\mathbb{E}[T] = h \cdot T_{\text{hit}} + (1 - h) \cdot T_{\text{miss}}(n), \quad (5)$$

where h is the hit rate achieved by a candidate weighting and $T_{\text{miss}}(n)$ is a linear fit of latency against prefetched token count n , estimated from miss-only observations. A single weight is selected per query type and applied at every K . We exclude the semantic query category from this search entirely, using a fixed, heavy-semantic default ($\alpha=0.90$, $\beta=0.10$). The grid search converges to $\alpha=0.35$, $\beta=0.65$ for both structural and multi-hop queries in every configuration we report, including every newly added model in this revision – we return to the consequence of this coincidence in §6.

3.5 Prefetch Policies

Four policies are compared under identical hardware conditions per model. **Cold** performs no proactive prefetching, though it still benefits passively from whatever chunks remain resident in the capacity-bounded cache from earlier accesses. **Cosine** prefetches the top- K chunks by raw cosine similarity at each access. **Graph** prefetches the top- K two-hop graph neighbors by fixed combined weight. **Adaptive** re-ranks the same candidates using the learned, query-type-specific weighting.

3.6 Hit-Aware Hardware Evaluation Design

Each evaluation trial proceeds in three steps. First, the primary chunk c_p is processed and its `past_key_values` K_A are computed and stored. Second, under a prefetch policy, the predicted chunks are processed to produce an extended KV state K_{A+P} ; this step’s cost is not charged to the trial, since in a deployed system this work happens ahead of and independent of the user’s query. Third, the secondary chunk c_s is served.

An earlier version of our evaluation harness unconditionally re-ran a full forward pass over c_s in this step, regardless of whether c_s was already present in K_{A+P} from the prefetch step. Under that measurement, a cache hit never cost less than a cache miss, so cold – the policy with the shortest resident context and therefore the cheapest possible attention cost – could only ever be measured as tied with or faster than any prefetching policy, independent of how effective that policy’s predictions actually were.

We correct this by charging a genuine cache hit only the cost of a cache lookup rather than a fresh forward pass: $T_{\text{hit}} = 0$ ms, determined by a lightweight, capacity-bounded LRU tracker per policy; on a miss, the forward pass over c_s against K_{A+P} is run and measured exactly as before. We set `max_new_tokens`= 1 throughout to isolate prompt processing latency from generation latency. This methodology, and the harness that implements it, is unchanged from our earlier report and is applied identically across every model, family, and dataset added in this revision.

4 Experimental Setup

4.1 Models, Hardware, and Coverage

This revision substantially widens model coverage relative to our earlier report. Table 1 summarizes exactly which policy $\times$ dataset combinations exist for each model; coverage is *not* uniform, and we report this candidly rather than presenting partial runs as if they were complete sweeps.

Table 1: Model $\times$ dataset $\times$ policy coverage in this revision. **C/Co/G/A** = Cold/Cosine/Graph/Adaptive. A dataset cell lists which of the four policies were run on that dataset for that model.

Model	Hardware	HotpotQA	2WikiMQA	MuSiQue & MultiFieldQA-en
Qwen2.5-1.5B-Instruct	RTX 5060 Ti	C/Co/G/A	C/Co/G/A	C/Co/G/A (both)
Qwen2.5-3B-Instruct	RTX 5060 Ti	C/Co/G/A	C/Co/G/A	C/Co/G/A (both)
Qwen2.5-7B-Instruct	RTX 5060 Ti	C/Co/G/A	C/Co/G/A	C/Co/G/A (both)
Llama-3.2-1B	Colab / lab GPU	—	—	C/Co/G/A on MuSiQue; C/Co/G only on MultiFieldQA-en
Llama-3.2-3B	Colab T4	C/Co/G	—	C/Co/G on MultiFieldQA-en only
Gemma-3-1B	RTX 5060 Ti	C/Co/G	C/Co/G	C/Co/G (both)

The Qwen2.5 family (1.5B/3B/7B-Instruct, float16) is now evaluated identically and completely: all four LongBench datasets, all four policies, on a single RTX 5060 Ti. This is our **primary**

evaluation and supersedes the separate model-scale sweep (HotpotQA only) and dataset sweep (1.5B only, Colab T4) reported earlier; we retain the earlier Colab T4 per-dataset breakdown for Qwen2.5-1.5B in Appendix B as complementary, hardware-independent evidence, since it isolates dataset effects on a second GPU rather than pooling across datasets as the new sweep does.

Llama-3.2-1B, Llama-3.2-3B, and Gemma-3-1B constitute a **cross-architecture generalization check**. Coverage here is partial, reflecting compute and API budget constraints rather than a deliberate experimental design choice: Llama-3.2-1B has adaptive results only for MuSiQue; Llama-3.2-3B and Gemma-3-1B have no adaptive results at all (cold/cosine/graph only). We treat these as directional evidence for whether the graph and adaptive policies’ qualitative behavior reproduces outside the Qwen family, not as a like-for-like scale comparison with the primary evaluation.

4.2 Datasets and Query Generation

Datasets are the same four LongBench (Bai et al., 2024) splits used previously: HotpotQA (Yang et al., 2018) (376 chunks), 2WikiMultiHopQA (Ho et al., 2020) (432 chunks), MuSiQue (Trivedi et al., 2022) (421 chunks), and MultiFieldQA-en (396 chunks). For each dataset we generate $N=600$ deterministic query pairs (RANDOM_SEED= 42), targeting a 25%/25%/50% semantic/structural/multi-hop split; the realized per-dataset counts are $n=173$ semantic, $n=140$ structural, and $n=287$ multi-hop in every configuration, since all datasets and models share the identical seeded query pairs. Cache capacity is fixed at $C_{\max}=16$ throughout. Configurations sweep $K \in \{3, 4, 5, 6, 7, 8, 10, 12, 14, 16\}$ wherever the full grid was run; some cross-architecture configurations use a reduced grid (Table 1).

4.3 Closed-Source Answer-Quality Evaluation

Separately from the latency measurements above, we ask a different question: does a prefetching policy’s choice of context (i.e., which chunks it selects as the top- K candidates) change the quality of the answer an LLM produces from that context, independent of caching? This uses closed-source models we cannot run locally, accessed via API: **Gemini 2.5 Flash-Lite** (Google AI API) and **Llama-3.3-70B** (via Groq). For each provider, we sample HotpotQA queries ($n=60$ target), supply each of the three context-selection policies’ (cosine/graph/adaptive) top- K chunk set as context, and score the generated answer against the gold answer with exact-match (EM) and token-level F1. We additionally sweep the graph’s top- M construction parameter ($M \in \{5, 8, 12, 20\}$) for Gemini to check sensitivity of the quality ranking to graph density; the Groq run uses $M=12$ only. The Groq configuration completed $n=53/60$ queries (partial, due to provider-side rate limiting/timeouts); we report this run with its realized n rather than padding or discarding it.

5 Results

5.1 Primary Evaluation: Qwen2.5 Family, All Four Datasets, RTX 5060 Ti

Table 2 summarizes the pooled (all four datasets) AUC-integrated latency and crossover behavior for all three Qwen2.5 scales. The headline result of this revision is the adaptive policy’s crossover: it is now **exactly $K=6$ at every one of the three scales**, and the advantage holds at every larger K tested in every case. This is earlier and more uniform than the $K=7$ crossover we reported previously on HotpotQA alone at 1.5B/3B, and it is the first time we have a same-crossover-point result across the full model-scale range including 7B.

The fixed-weight graph policy’s overall crossover is likewise uniform once pooled across all four datasets: **$K=14$ at 1.5B, 3B, and 7B alike**, holding at every larger K in every case. This is a

cleaner pattern than the scale-dependent recession we noted in an earlier, HotpotQA-only version of this sweep, where the graph’s crossover appeared to recede from $K=14$ to $K=10$ with scale; pooling across the full dataset set removes that apparent trend, which we now read as a dataset-specific artifact of HotpotQA rather than a genuine scale effect.

Table 2: Qwen2.5 family, all four LongBench datasets pooled, RTX 5060 Ti. AUC = integrated mean latency (ms) across the full K sweep, lower is better. Crossover K = smallest K at which the policy significantly ($p < 0.05$) beats cosine; “holds” indicates the advantage is sustained at every larger tested K .

Scale	Policy	AUC	Crossover K	Holds	Struct. K	Multi-hop K
1.5B	Cosine	149.35	—	—	—	—
	Graph	179.28	14	Yes	3	16
	Adaptive	122.55	6	Yes	3	10 (No)
3B	Cosine	180.85	—	—	—	—
	Graph	216.62	14	Yes	3	None
	Adaptive	145.62	6	Yes	3	10 (Yes)
7B	Cosine	235.10	—	—	—	—
	Graph	277.42	14	Yes	3	16
	Adaptive	187.55	6	Yes	3	10 (Yes)

At every scale, cold trails badly on integrated latency (411–637 ms AUC, roughly $2.7\text{--}3.4\times$ cosine’s own AUC), confirming that some form of prefetching remains worthwhile at every scale we test even before considering graph or adaptive. Full per- K breakdowns (all ten tested K values, all four policies, all three query-type subgroups) for all three scales are reported in Appendix C.

5.2 Cross-Architecture Generalization: Llama-3.2 and Gemma-3

Table 3 reports the same summary statistics for the three non-Qwen models, with the coverage caveats of Table 1 in mind.

Table 3: Cross-architecture generalization summary. Coverage is partial per Table 1; AUC and crossover are computed over whichever K -grid and dataset pool was actually run for that model (see §4).

Model	Policy	AUC	Crossover K	Holds	Struct. K	Multi-hop K
Llama-3.2-1B	Cosine	124.45	—	—	—	—
	Graph	138.80	10	Yes	3	None
	Adaptive [†]	98.80	6	Yes	3	14 (Yes)
Llama-3.2-3B (no adaptive run)	Cosine	322.55	—	—	—	—
	Graph	360.32	10	Yes	3	None
Gemma-3-1B (no adaptive run)	Cosine	254.42	—	—	—	—
	Graph	273.40	10	Yes	3	10 (No)

[†]Adaptive results for Llama-3.2-1B use only the MuSiQue subset ($n=600$); cold/cosine/graph rows are pooled across MuSiQue and MultiFieldQA-en (see Table 1).

Two patterns stand out. First, **the structural-query crossover is exactly $K=3$ in every one of the six models we evaluate** – Qwen2.5 at 1.5B/3B/7B, Llama-3.2 at 1B/3B, and Gemma-

3-1B – with the advantage holding at every larger K in every case and no exceptions. This is, across this entire body of work, the single most reproducible effect we have observed, and it is now confirmed across three unrelated model families rather than one. Second, the fixed-weight graph policy’s *overall* crossover is $K=10$ for all three non-Qwen models, consistently earlier than the $K=14$ we observe across all three Qwen scales; we do not have a confident explanation for this family-level difference and flag it as an open question in §6 rather than speculating further. As with Qwen2.5-7B in our earlier report, Llama-3.2-3B and Gemma-3-1B (both untested with adaptive) show cosine ranking ahead of graph on aggregate AUC, which reflects graph’s losses at small K rather than its performance at the K values one would actually deploy at $K \geq 10$.

5.3 Query-Type Analysis

Structural queries remain the most robust source of advantage. Beyond the universal $K=3$ crossover noted above, the magnitude of the structural effect scales with model size within the Qwen family: at $K=16$, the adaptive policy’s structural advantage over cosine is +16.7 ms at 1.5B, +20.4 ms at 3B, and +24.8 ms at 7B, a monotonic increase that mirrors our earlier finding that a larger model’s higher fixed per-token cost makes the absolute latency saved by skipping a re-encode grow with scale. The cross-architecture models show smaller absolute structural deltas at $K=16$ (Gemma-3-1B +11.3 ms, Llama-3.2-1B +5.3 ms graph vs. cosine), consistent with their smaller parameter counts, but the direction and significance are identical.

Semantic queries favor cosine everywhere, by construction, exactly as before: a semantic query is defined as the single highest-cosine-similarity chunk to the currently active one, so cosine ranking is guaranteed to retrieve it, and no learned edge weighting changes that. This holds without exception across all six models in this revision.

Multi-hop queries remain the noisiest category. Within Qwen2.5, the adaptive policy’s multi-hop crossover is $K=10$ at all three scales, but only *sustained* at every larger K for 3B and 7B – at 1.5B it becomes significant at $K=10$ and again at $K=14/K=16$ but is not uniformly significant in between, matching the pattern of comparative instability in this subgroup that we flagged previously. Cross-architecture, Llama-3.2-1B’s adaptive multi-hop advantage crosses over later, at $K=14$, but does hold at every larger K thereafter – directionally consistent with, but slower to emerge than, the Qwen pattern. Neither Llama-3.2-3B nor Gemma-3-1B’s fixed graph policy ever reaches significance on multi-hop queries (Gemma reaches significance only transiently at $K=10$), reinforcing that the multi-hop advantage is substantially a property of the *adaptive* policy rather than the fixed-weight graph, since the fixed graph alone struggles to realize it consistently even within Qwen (3B’s fixed graph also never reaches multi-hop significance, Table 2).

5.4 Hit-Rate Trends with K

We previously reported that the adaptive policy’s hit rate plateaus exactly at 77.5% for both 1.5B and 3B from $K=10$ through $K=16$ on a HotpotQA-only sweep, with no further change. **Pooled across all four datasets, this plateau does not reproduce:** adaptive hit rate at 1.5B continues to climb from 78.2% ($K=10$) to 79.8% ($K=12$), 81.5% ($K=14$), and 82.7% ($K=16$); the pattern at 3B (78.2% $\rightarrow$ 80.5% $\rightarrow$ 83.1% $\rightarrow$ 84.4%) and 7B (78.2% $\rightarrow$ 80.5% $\rightarrow$ 83.1% $\rightarrow$ 84.4%) is nearly identical in shape. We revise our earlier claim accordingly: the apparent saturation was specific to the smaller, single-dataset candidate pool available under HotpotQA alone, and does not hold once the two-hop candidate pool is drawn from the full, larger, four-dataset evaluation. We note this revision explicitly rather than silently updating the number, in keeping with the methodological transparency this paper otherwise argues for (§3.6).

5.5 Robustness to the Maximum-Degree Cap

As previously reported, the 1.5B and 3B adaptive results use a maximum in-degree of 10; the 7B graph-only baseline in our earliest report used 100. Re-running the 1.5B and 3B adaptive configurations with the cap raised to 100 produced no material change in AUC, crossover point, or the query-type pattern above. Graph construction logs across every newly added model and dataset in this revision (Llama, Gemma) show realized average degree in the same 27–29 band regardless of corpus, confirming the cap remains non-binding at $M=20$ throughout.

5.6 Closed-Source Answer-Quality Evaluation

Table 4 reports F1 and EM for both API providers. This is the first check in this line of work of whether a policy’s latency advantage says anything about the quality of the answer it helps produce, and the result is, candidly, the opposite ranking from every latency table above.

Table 4: Answer-quality (F1/EM) by context-selection policy and top- M , HotpotQA, closed-source models. Groq’s row is partial ($n=53/60$); all other rows are $n=60/60$.

Provider	top- M	n	Cosine F1	Graph F1	Adaptive F1	Cosine EM	Graph EM	Adaptive EM
Gemini (Flash-Lite)	5	60	0.363	0.349	0.262	0.233	0.217	0.200
Gemini (Flash-Lite)	8	60	0.371	0.333	0.296	0.233	0.233	0.200
Gemini (Flash-Lite)	12	60	0.376	0.332	0.270	0.250	0.233	0.183
Gemini (Flash-Lite)	20	60	0.374	0.332	0.305	0.233	0.217	0.200
Groq (Llama-3.3-70B)	12	53*	0.371	0.375	0.325	0.189	0.189	0.132

*53 of 60 targeted queries completed; the remaining 7 did not return a scorable response from the Groq endpoint within the evaluation window.

On Gemini, cosine leads graph and adaptive on F1 at every one of the four top- M settings we tested, and leads or ties on EM at every setting. Adaptive is the weakest of the three policies on F1 at every Gemini setting. On Groq’s Llama-3.3-70B, the ranking is only partially different: graph edges out cosine on F1 by a small margin at the single top- M we tested (0.375 vs. 0.371) and ties it on EM (0.189 each), but adaptive again trails both alternatives by a wide margin on both metrics (F1 0.325, EM 0.132). Across both providers and all five configurations, **adaptive is never the best-performing policy on answer quality**, despite being the strongest policy by a wide margin on latency in every experiment in §5.1 and §5.2.

We discuss the likely explanation and the appropriate caveats in §6; the short version is that this evaluation is preliminary ($n=53$ –60, one dataset, one query-type distribution not broken out by subgroup) and should be read as a first flag rather than a settled result.

6 Discussion

Why the fixed-weight graph is not enough on its own. The fixed weighting $\alpha=0.70$, $\beta=0.30$ is a single compromise that must serve semantic, structural, and multi-hop queries simultaneously. Pooled across all four datasets and all three Qwen scales, its overall crossover recedes to $K=14$ – later than the adaptive policy’s uniform $K=6$ – and it still never reaches significance on multi-hop queries at 3B. The consistent $K=6$ adaptive crossover, now confirmed at every Qwen scale, remains the strongest evidence in this paper that query-type-aware weighting is necessary rather than a marginal refinement.

A family-level difference we cannot yet explain. The fixed graph’s overall crossover is uniformly $K=14$ across all three Qwen2.5 scales but uniformly $K=10$ across all three non-Qwen models (Llama-3.2-1B/3B, Gemma-3-1B). Since chunking, embeddings, and graph construction are identical across families, this difference must originate in each family’s own attention cost profile or hit-rate response to the same graph structure. We flag this as an open question for future work (§8) rather than speculate further with the coverage we currently have.

The shared structural/multi-hop weight. As before, the grid search selects the same ($\alpha=0.35, \beta=0.65$) pair for both structural and multi-hop queries in every configuration, including every newly added model. Given that roughly half of our multi-hop queries are themselves close structural jumps, a weighting tuned primarily against structural queries plausibly serves the structural half of multi-hop well while under-serving its semantic half, which remains a plausible explanation for why the multi-hop advantage is real but noisier than the structural one – and this pattern is now visible cross-architecturally as well (Llama-3.2-1B’s later, $K=14$, multi-hop crossover).

Latency and answer quality are not the same objective. The closed-source QA-quality check in §5.6 is, in our view, the most important addition in this revision precisely because it complicates the paper’s main story rather than reinforcing it. The graph and adaptive policies are trained and selected purely to minimize expected *cache* latency (Eq. 5 in our method); nothing in that objective rewards retrieving the single most semantically relevant chunk if a structurally adjacent chunk is more likely to be re-accessed soon. For a downstream QA model that must produce a factually grounded answer from whatever context it is given, the two goals can diverge: a graph- or adaptive-selected top- K set that is excellent for co-access prediction can still omit the one chunk that actually contains the answer, if that chunk happens to be more semantically distant than an adjacent one. Cosine, by contrast, is directly optimized for semantic relevance to the query and therefore has a structural advantage (in the ordinary sense of the word) for this task that has nothing to do with our chunk graph. The adaptive policy’s consistently *worst* F1/EM across both providers is consistent with this: it is the policy that most aggressively de-weights semantic similarity ($\beta=0.65$ for structural/multi-hop queries), which is exactly what we would expect to cost it the most on a quality metric that rewards semantic relevance directly. We do not read this as undermining the latency results – the two evaluations measure genuinely different things – but it is a real trade-off that any deployment of this system should weigh, and we did not have this evidence when we first wrote this paper.

7 Limitations

Cross-architecture coverage is partial by necessity, not design. Table 1 is not a balanced grid: Llama-3.2-1B has adaptive results for only one of two tested datasets; Llama-3.2-3B and Gemma-3-1B have no adaptive results at all; Llama-3.2-3B’s non-adaptive coverage spans only two of the four LongBench datasets. Every cross-architecture claim in §5.2–§5.3 should be read with this in mind – we are confident in the direction of the effects we report, less confident that their exact magnitudes would hold under full coverage.

The QA-quality evaluation is small and single-dataset. Table 4 covers one dataset (HotpotQA), $n=53$ –60 queries per configuration, and two providers. We did not break these results out by query type (semantic/structural/multi-hop), which §6’s explanation would predict matters a great deal – we would expect the quality gap to be smallest or reversed on structural queries and largest on semantic queries, but we do not yet have the per-subgroup numbers to check this. Groq’s run is additionally partial ($n=53/60$).

The two evaluations still do not jointly sweep every axis. Our primary evaluation sweeps

model scale and dataset, jointly, within one family (Qwen2.5) on one GPU. Our cross-architecture check sweeps family with a reduced dataset/policy grid. Our QA-quality check sweeps provider and top- M on one dataset with no latency measurement at all. No single evaluation in this paper jointly sweeps model family, model scale, dataset, and answer quality.

A revised claim, not a new bug. §5.4’s hit-rate plateau revision is a genuine correction to an earlier reported finding, not a new methodological defect; we flag it here rather than only in the results section because it is a limitation of the earlier, narrower evaluation rather than of the corrected hit-aware harness itself, which is unchanged.

Static graph; structural heuristic. As before, the hybrid graph is constructed once per corpus and does not adapt to observed co-retrieval patterns at runtime, and the structural score $1/(1 + |i - j|)$ does not capture long-range structural relationships such as introduction–conclusion cross-references.

8 Future Work

The most immediate priority is completing adaptive coverage for Llama-3.2-3B and Gemma-3-1B, and completing the remaining dataset/policy cells in Table 1 for Llama-3.2-1B, so that the cross-architecture claims in §5.2 rest on the same full grid as the Qwen2.5 primary evaluation. Second, the family-level difference in the fixed graph’s crossover point ($K=14$ for Qwen vs. $K=10$ for the other three families, §6) deserves direct investigation, ideally by instrumenting each family’s own attention cost per resident token rather than inferring it indirectly from crossover points. Third, and in our view most important, the QA-quality check in §5.6 should be expanded: more datasets, larger n , and a per-query-type breakdown, to test directly whether the quality gap we observe is concentrated on semantic queries as §6 predicts, or is more uniform than that account suggests. If it is concentrated on semantic queries, a natural next step is a hybrid deployment policy that uses adaptive/graph prefetching for cache efficiency but falls back to cosine-selected context specifically for queries classified as semantic, capturing the latency benefit without the quality cost. Finally, production integration by replacing the explicit `past_key_values` mechanism used here with vLLM’s native prefix caching, combined with LMCatch’s CacheBlend-style blending for non-prefix reuse (Liu et al., 2024b), remains an open and, we think, now more urgent goal, since it would let the QA-quality check and the latency check finally run on the exact same served requests.

9 Conclusion

We presented a hybrid chunk graph and an adaptive, query-type-specific extension for proactive KV cache prefetching in RAG systems, evaluated under a corrected, hit-aware latency measurement. In this revision, we widen the evaluation along three further axes: model scale (now a complete Qwen2.5 1.5B/3B/7B sweep across all four LongBench datasets, rather than separate scale and dataset sweeps), model family (Llama-3.2 and Gemma-3, with partial coverage), and, for the first time, downstream answer quality via two closed-source models. The adaptive policy’s advantage over cosine on latency is now the most consistent result we have: significant from $K=6$ onward at every Qwen scale, holding at every larger K . The structural-query crossover at exactly $K=3$ reproduces without exception across all six models and three model families we test, the single most robust finding in this body of work. Set against this, our closed-source quality check finds that cosine similarity matches or beats graph and adaptive on answer F1/EM in four of five configurations, and that adaptive – the strongest policy on latency – is the weakest policy on quality in every configuration tested. We report this candidly: a policy that is good at predicting which chunks will

be re-accessed soon is not thereby good at selecting which chunks best answer the question, and we think this tension, not just the latency numbers, is the right thing for this line of work to study next.

References

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137, 2024.
- Tien-Fu Chen and Jean-Loup Baer. An effective on-chip preloading scheme to reduce data access penalty. In *Proceedings of the International Conference for High Performance Computing and Communications*, pages 176–186, 1995.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- Gemini Team, Google. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.
- Gemma Team, Google DeepMind. Gemma 3 technical report. *arXiv preprint*, 2025.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, 2020.
- Radeen Hu, Liang Tan, Jiajun Zou, and Xinyun Luo. G-RAG: Knowledge expansion in material science. *arXiv preprint arXiv:2408.17545*, 2024.
- Chao Jin, Zili Ye, Ruiting Li, Yiling Huang, Yibo Ding, Fan Zhang, Xuanzhe Du, and Xin Jin. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Meta AI. Llama 3 model card / technical overview. 2024.
- Yuhan Liu, Hanchen Li, Kuntai Du, Jiayi Yao, Yihua Cheng, Yuyang Huang, Shan Lu, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. *arXiv preprint arXiv:2310.07240*, 2024.

- Yuhan Liu, Hanchen Li, Kuntai Du, Jiayi Yao, Yihua Cheng, Yuyang Huang, Shan Lu, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. LMCache: Enabling efficient prefix caching for LLM inference. *arXiv preprint arXiv:2404.09947*, 2024.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, 2018.

A Hyperparameter Configuration

Table 5: Full hyperparameter configuration used in all experiments, including every newly added model.

Parameter	Value	Rationale
Chunk size	500 chars	Fits embedding model token limit
Chunk overlap	50 chars	Prevents sentence truncation at boundaries
Cache capacity	16 chunks	Constrained VRAM KV cache budget
Semantic construction	Top- M nearest-neighbor, $M=20$	Guarantees per-node semantic coverage independent of the corpus’s similarity distribution
Secondary semantic threshold τ	0.50	Retained as a supplementary gate alongside top- M
Semantic weight α (fixed graph)	0.70	Semantic signal is the primary relevance driver
Structural weight β (fixed graph)	0.30	Structural signal provides complementary context
Adaptive weights (semantic, fixed)	$\alpha=0.90, \beta=0.10$	Not learned – see §3.4
Adaptive weights (structural, learned)	$\alpha=0.35, \beta=0.65$	Grid search, expected-latency objective; identical across every model in this revision
Adaptive weights (multi-hop, learned)	$\alpha=0.35, \beta=0.65$	Same value as structural in every configuration tested
Two-hop candidate score	$0.9 \cdot \min(w_1, w_2) + 0.1 \cdot \max(w_1, w_2)$	Bottleneck-scored but distinguishes hop-strength pairs
Max node in-degree	100 (10 tested, no material difference)	Limits graph density
Hit latency T_{hit}	0 ms	A cache hit costs a metadata lookup, not a forward pass
Embedding model	all-MiniLM-L6-v2	384-dimensional
LLM models (latency evaluation)	Qwen2.5-{1.5B,3B,7B}-Instruct; Llama-3.2-{1B,3B}; Gemma-3-1B	See Table 1 for exact coverage per model
LLM models (QA-quality evaluation)	Gemini 2.5 Flash-Lite; Llama-3.3-70B (via Groq)	Closed-source / API-only, see §4.3
Precision	float16	Local models, matched across hardware platforms
max_new_tokens (latency eval)	1	Isolates prompt processing from generation latency
Query distribution	25%/25%/50% target (realized $n=173/140/287$)	Reflects a realistic RAG workload distribution
N per configuration (latency)	600 pairs	Statistical power for per-subgroup analysis
n per configuration (QA quality)	60 target (53–60 realized)	API cost/rate-limit constrained
RANDOM_SEED	42	Full reproducibility; identical pairs across all configurations

B Complementary Evidence: Colab T4, Qwen2.5-1.5B, Per-Dataset Breakdown

We retain the following earlier result as complementary, hardware-independent evidence for dataset generalization: on a Colab T4, sweeping Qwen2.5-1.5B across all four LongBench datasets individually (rather than pooled), the fixed graph policy’s overall advantage reached significance on only one of four datasets (MuSiQue, $K=14$); the adaptive policy crossed over at exactly $K=8$ on all four datasets, holding at every larger K in every case. This per-dataset granularity is not available for the newer, pooled Qwen2.5 sweep in §5.1, since that evaluation’s underlying reports pool across datasets by construction; we present both as complementary rather than attempting to force one into the other’s format.

C Full Per- K Results, Qwen2.5 Family

Figures 1–3 reproduce the full six-panel evaluation (mean latency vs. K , hit rate vs. K , relative speedup vs. cosine, FLOPs-saved estimate vs. K , cosine–graph delta by query type, and cosine–adaptive delta by query type) for each Qwen2.5 scale, generated directly from the pooled four-dataset evaluation logs referenced in §5.1. Tables 6–8 give the same sweep as plain numbers, at every one of the ten tested K values rather than a selected subset, so the figures can be checked against exact latency and hit-rate readings.

Table 6: Qwen2.5-1.5B, full per- K sweep, pooled across all four LongBench datasets, RTX 5060 Ti. Latency in ms (lower is better); hit rate as a percentage.

K	Cold (ms)	Cosine (ms)	Graph (ms)	Adaptive (ms)	Cos. hit%	Graph hit%	Adpt. hit%
3	31.59	13.98	22.54	19.32	57.2	31.1	40.5
4	31.92	13.18	20.57	16.49	60.3	38.7	50.5
5	31.82	12.86	18.71	13.57	61.2	44.7	58.9
6	31.55	12.22	16.95	11.43	62.4	49.9	65.8
7	30.51	11.46	15.41	10.32	63.5	53.0	69.0
8	31.74	11.85	14.29	9.10	64.2	58.2	73.1
10	31.93	11.39	11.87	7.50	66.5	64.5	78.2
12	31.43	10.40	10.88	7.03	68.3	67.9	79.8
14	31.82	10.40	10.11	6.27	68.9	70.1	81.5
16	31.86	10.51	9.22	5.81	69.3	72.3	82.7

Table 7: Qwen2.5-3B, full per- K sweep, pooled across all four LongBench datasets, RTX 5060 Ti.

K	Cold (ms)	Cosine (ms)	Graph (ms)	Adaptive (ms)	Cos. hit%	Graph hit%	Adpt. hit%
3	37.28	17.17	28.63	24.47	57.2	31.1	40.5
4	37.58	16.03	25.23	20.24	60.3	38.7	50.5
5	37.03	15.60	22.53	16.85	61.2	44.7	58.9
6	37.67	15.44	20.43	13.68	62.4	49.9	65.8
7	37.18	14.79	18.96	12.41	63.3	53.2	69.2
8	37.23	14.39	17.25	10.85	64.2	58.2	73.1
10	37.12	13.36	14.16	8.99	66.5	64.5	78.2
12	37.19	12.52	12.89	7.99	68.3	67.9	80.5
14	37.46	12.35	11.93	6.84	68.9	70.0	83.1
16	37.17	12.36	11.34	6.27	69.3	72.0	84.4

Table 8: Qwen2.5-7B, full per- K sweep, pooled across all four LongBench datasets, RTX 5060 Ti.

K	Cold (ms)	Cosine (ms)	Graph (ms)	Adaptive (ms)	Cos. hit%	Graph hit%	Adpt. hit%
3	50.34	23.21	36.82	31.84	57.2	31.1	40.5
4	50.46	21.02	32.55	26.15	60.3	38.7	50.5
5	49.76	20.41	29.17	21.63	61.2	44.7	58.9
6	49.80	19.81	26.34	18.05	62.4	49.9	65.8
7	50.77	19.64	25.64	16.96	63.5	53.0	69.0
8	48.78	18.70	21.68	13.83	64.2	58.2	73.1
10	48.27	17.55	18.00	11.28	66.5	64.5	78.2
12	48.40	16.16	16.34	10.08	68.3	67.9	80.5
14	48.17	15.78	15.09	8.71	68.9	70.0	83.1
16	48.06	15.59	13.93	7.95	69.3	72.0	84.4

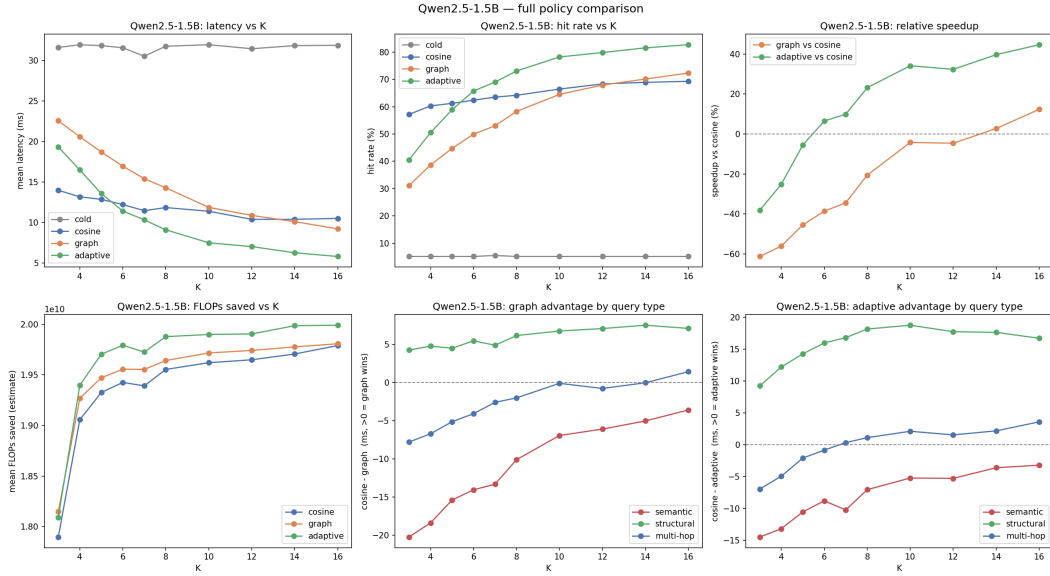


Figure 1: Qwen2.5-1.5B, all four datasets pooled, RTX 5060 Ti.

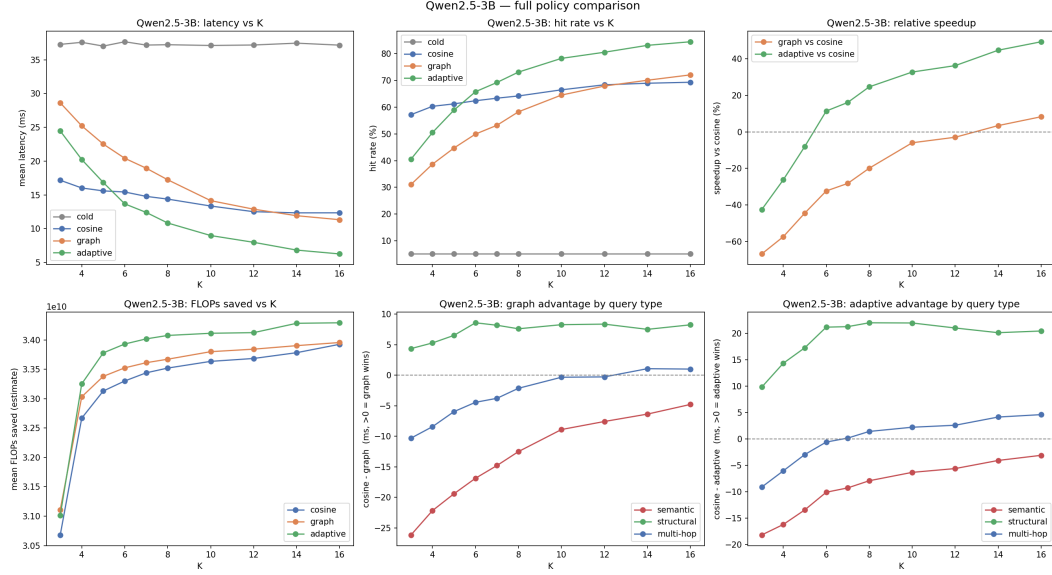


Figure 2: Qwen2.5-3B, all four datasets pooled, RTX 5060 Ti.

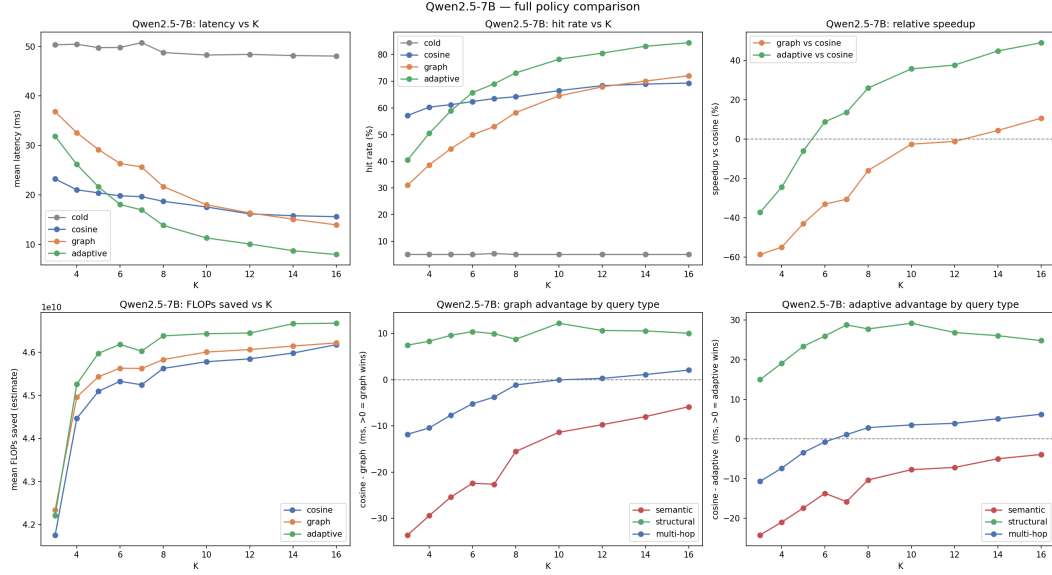


Figure 3: Qwen2.5-7B, all four datasets pooled, RTX 5060 Ti.

D Cross-Architecture and Cross-Model Figures

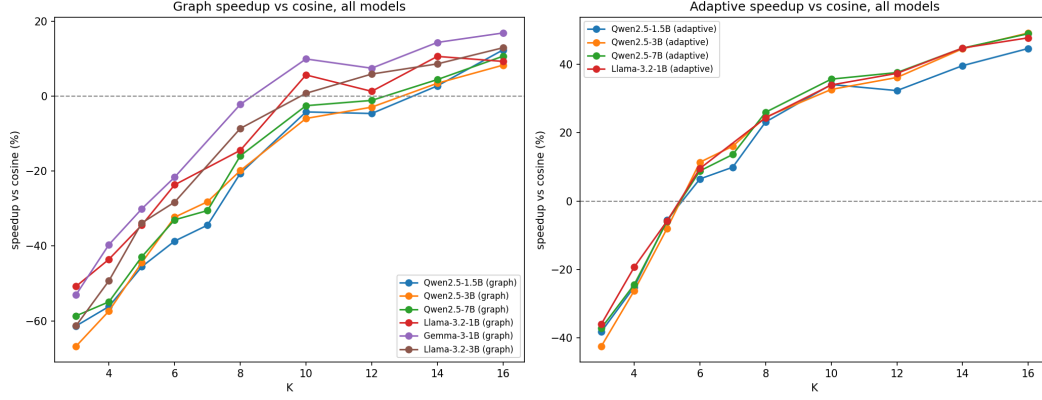


Figure 4: Cross-model overview: latency and hit-rate behavior compared across all six models evaluated in this revision.

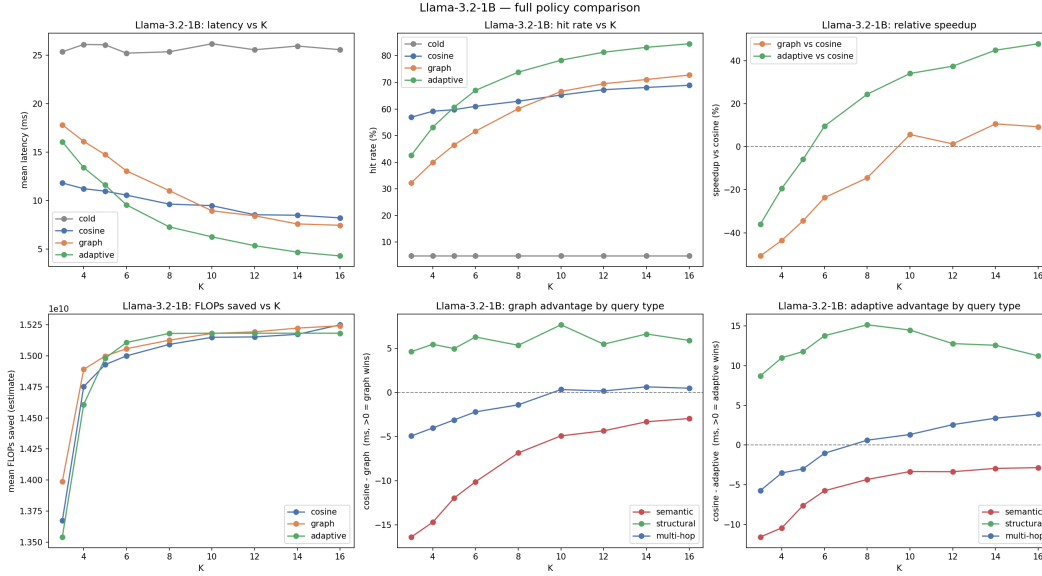


Figure 5: Llama-3.2-1B (MuSiQue + MultiFieldQA-en pooled; adaptive on MuSiQue only).

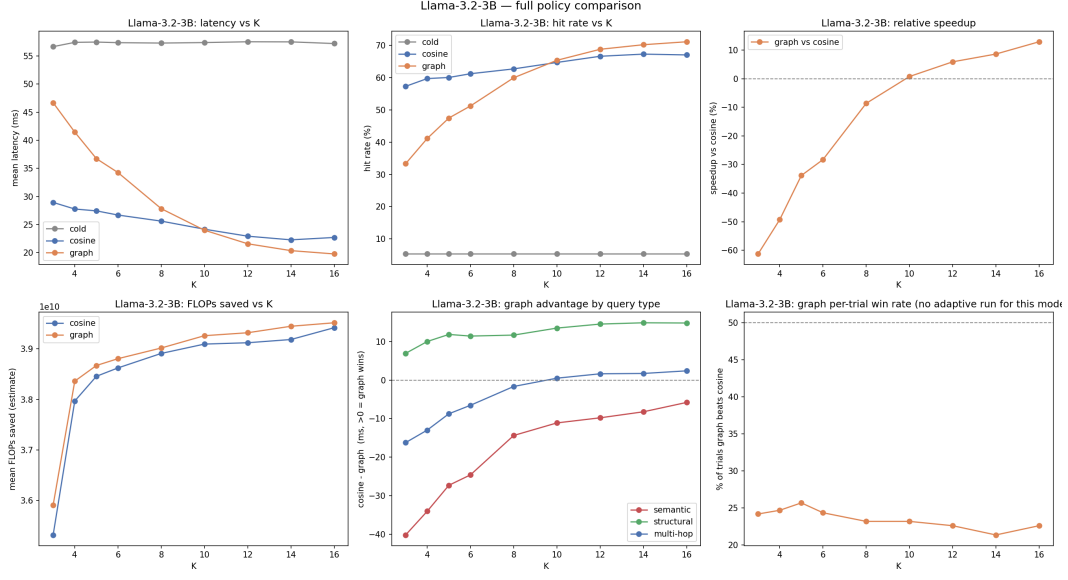


Figure 6: Llama-3.2-3B (HotpotQA + MultiFieldQA-en pooled; no adaptive run).

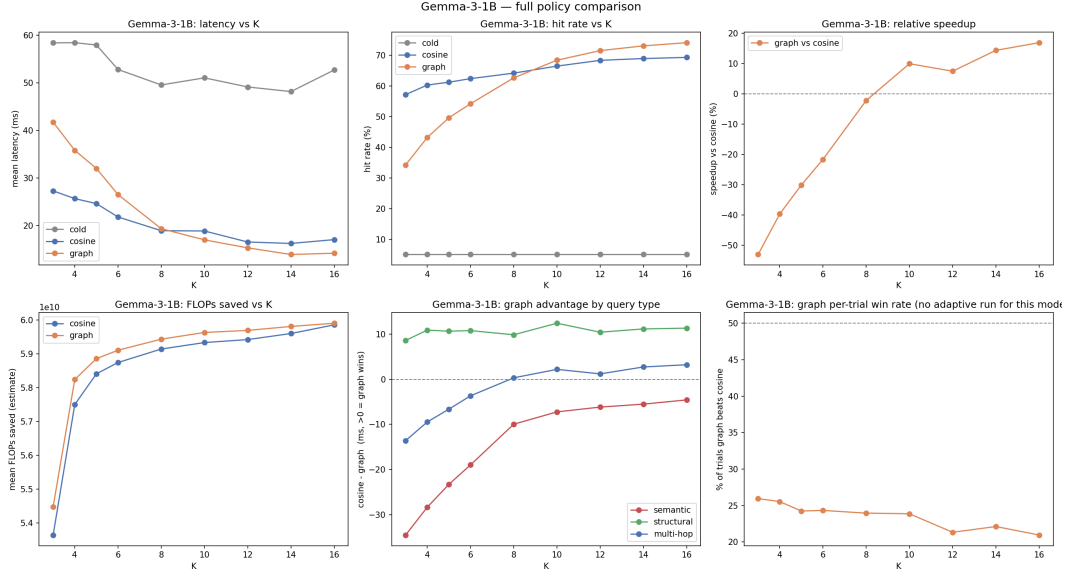


Figure 7: Gemma-3-1B (all four datasets pooled; no adaptive run).

E Closed-Source Answer-Quality Evaluation Figures

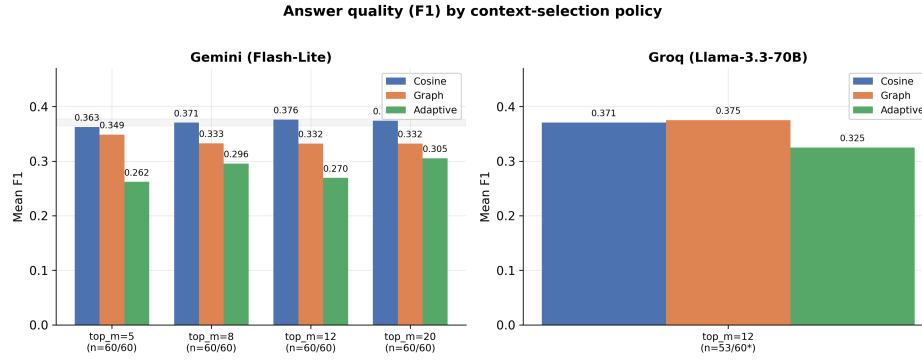


Figure 8: Gemini 2.5 Flash-Lite: F1 by policy across top- M settings.

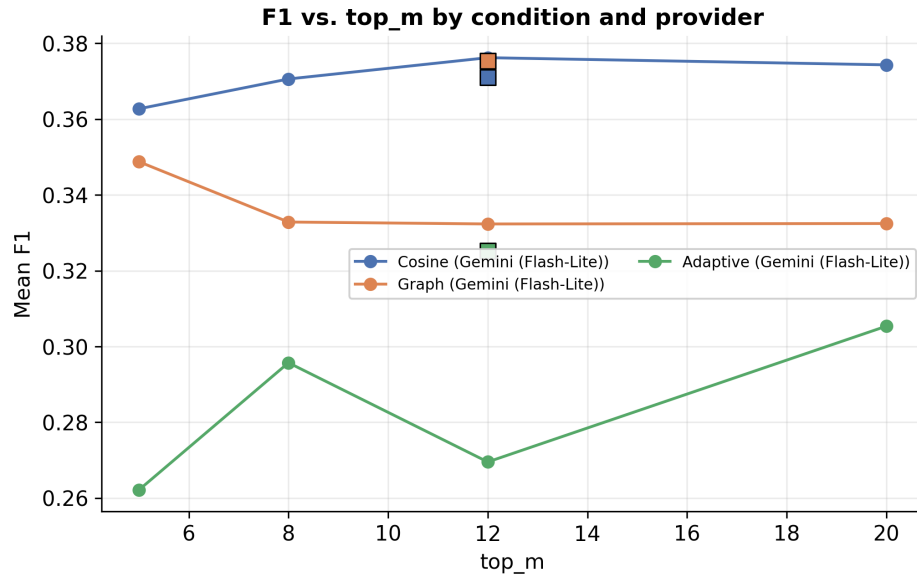


Figure 9: Sensitivity of the quality ranking to the graph’s top- M construction parameter.

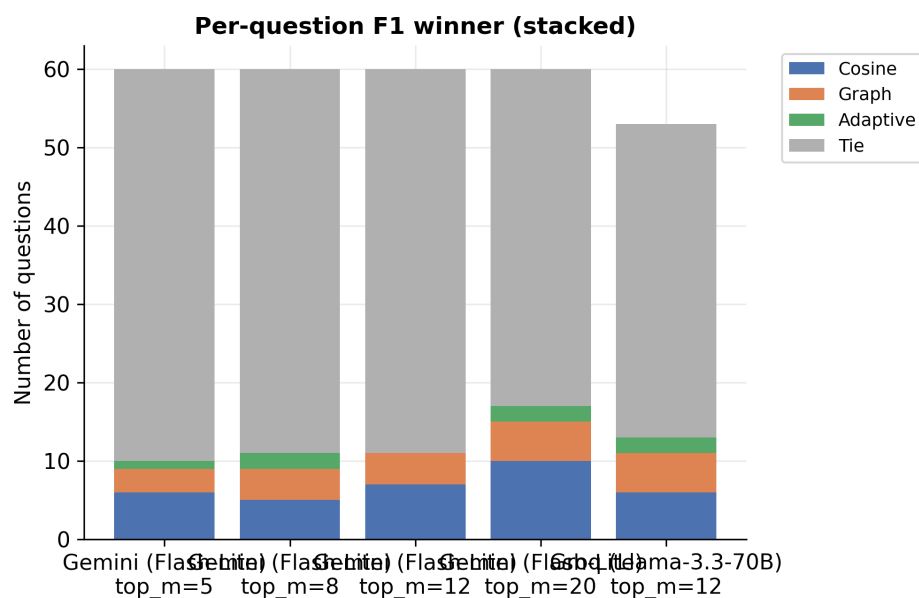


Figure 10: Per-query win counts by policy, Gemini 2.5 Flash-Lite.

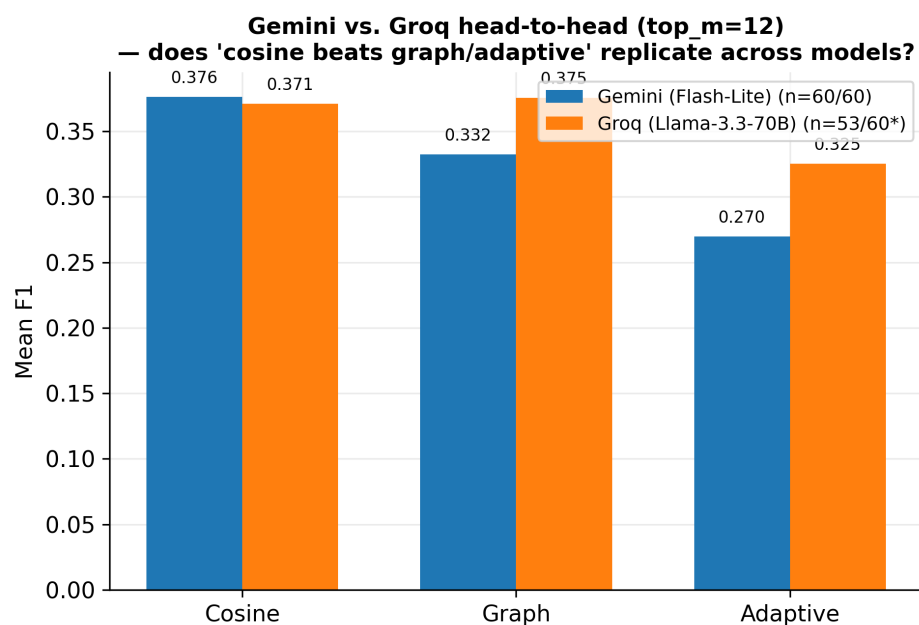


Figure 11: Head-to-head comparison of the quality ranking between Gemini 2.5 Flash-Lite and Llama-3.3-70B (Groq).